



```
In [ ]: !pip -q install transformers accelerate sentencepiece
```

```
In [ ]: from huggingface_hub import login

login()
```

```
In [ ]: from google.colab import userdata

# Retrieve Hugging Face token from Colab Secret Userdata
hf_token = userdata.get('token2')

# Set the token as an environment variable for Hugging Face
import os
os.environ["token2"] = hf_token

print("Hugging Face token loaded successfully.")
```

Hugging Face token loaded successfully.

Now that the token is set up, let's load the Gemma 3 model. We will use the `transformers` library for this.

```
In [ ]: import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "google/gemma-3-4b-it" # Using Gemma 3 4B IT version

tokenizer = AutoTokenizer.from_pretrained(model_id, token=hf_token)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16, # Use bfloat16 for better performance on
compatible GPUs
    device_map="auto", # Automatically maps the model to available
devices (e.g., GPU)
    token=hf_token # Pass the token explicitly
)
```

```
config.json: 0%|          | 0.00/855 [00:00<?, ?B/s]

tokenizer_config.json: 0.00B [00:00, ?B/s]

tokenizer.json: 0%|          | 0.00/33.4M [00:00<?, ?B/s]

added_tokens.json: 0%|          | 0.00/35.0 [00:00<?, ?B/s]

special_tokens_map.json: 0%|          | 0.00/662 [00:00<?, ?B/s]

`torch_dtype` is deprecated! Use `dtype` instead!

model.safetensors.index.json: 0.00B [00:00, ?B/s]

Downloading (incomplete total...): 0.00B [00:00, ?B/s]

Fetching 2 files: 0%|          | 0/2 [00:00<?, ?it/s]

Loading weights: 0%|          | 0/883 [00:00<?, ?it/s]

generation_config.json: 0%|          | 0.00/215 [00:00<?, ?B/s]
```

The model and tokenizer have been loaded. You can now use them to generate text.

```
In [ ]: !pip install -q torchinfo
```

```
In [ ]: import torchinfo

# Display model summary
torchinfo.summary(model)
```

```
=====
Layer (type:depth-idx)                               Param #
=====
Gemma3ForConditionalGeneration                       --
├─Gemma3Model: 1-1                                    --
│   └─SiglipVisionModel: 2-1                          --
│       └─SiglipVisionTransformer: 3-1
416,866,032
│   └─Gemma3MultiModalProjector: 2-2
2,949,120
│       └─Gemma3RMSNorm: 3-2                            1,152
│       └─AvgPool2d: 3-3                                --
│       └─Gemma3TextModel: 2-3                          --
│           └─Gemma3TextScaledWordEmbedding: 3-4
671,252,480
│           └─ModuleList: 3-5
3,209,008,128
│               └─Gemma3RMSNorm: 3-6                    2,560
│               └─Gemma3RotaryEmbedding: 3-7            --
└─Linear: 1-2
671,252,480
=====
Total params: 4,971,331,952
Trainable params: 4,971,331,952
Non-trainable params: 0
=====
```

Now, let's use the model to generate text based on your query.

```
In [ ]: input_text = "2026년 5월 23일 하이닉스 주가는 "  
  
# Tokenize the input text  
input_ids = tokenizer(input_text, return_tensors="pt").to(model.device)  
  
# Generate text using the model  
output_tokens = model.generate(**input_ids, max_new_tokens=10,  
num_return_sequences=1)  
  
# Decode the generated tokens  
generated_text = tokenizer.decode(output_tokens[0],  
skip_special_tokens=True)  
  
print(generated_text)
```

2026년 5월 23일 하이닉스 주가는 목표주가와 분석가 의견을 정리했습니다

To make the model behave more like a chatbot, we need to format the input in a conversational style. Here's how you can do that with a prompt template.

```

In [ ]: def chat_with_gemma(prompt, max_new_tokens=100):
    chat_template = [
        {"role": "user", "content": prompt},
    ]
    formatted_prompt = tokenizer.apply_chat_template(chat_template,
    tokenize=False, add_generation_prompt=True)

    input_ids = tokenizer(formatted_prompt,
    return_tensors="pt").to(model.device)
    output_tokens = model.generate(**input_ids,
    max_new_tokens=max_new_tokens, num_return_sequences=1)
    generated_text = tokenizer.decode(output_tokens[0],
    skip_special_tokens=True)

    # The generated_text will include the input prompt, so we need to
    remove it
    # to get only the model's response.
    # The exact parsing might depend on the model's output, but
    generally
    # we can look for the beginning of the assistant's response.
    try:
        # Attempt to find the start of the assistant's response after
        the user message
        # This is a common pattern for chat models
        assistant_start_tag = "<start_of_turn>model\n"
        start_index = generated_text.find(assistant_start_tag)
        if start_index != -1:
            response = generated_text[start_index +
            len(assistant_start_tag):].strip()
        else:
            # Fallback if the specific tag is not found, or if the model
            format is different
            # This might be less robust but tries to get the part after
            the initial prompt.
            response = generated_text.replace(formatted_prompt,
            "").strip()
    except Exception:
        response = generated_text # In case of parsing issues, return
        full text

    return response

# Example usage:
user_query = "2026년 5월 23일 하이닉스 주가는"
response = chat_with_gemma(user_query)
print(f"User: {user_query}")
print(f"Gemma: {response}")

```

User: 2026년 5월 23일 하이닉스 주가는

Gemma: user

2026년 5월 23일 하이닉스 주가는

model

2026년 5월 23일 하이닉스 주가에 대한 정보는 현재로서는 예측이 불가능합니다. 주가는 다양한 요인에 따라 변동하기 때문에 미래의 가격을 정확하게 예측하는 것은 어렵습니다.

하지만 현재 시점에서 하이닉스 주가에 영향을 미칠 수 있는 요인들을 고려하여 분석해 볼 수 있습니다.

****하이닉스(HLSI) 관련 주요 요인:****

* **

You can now call the `chat_with_gemma` function with your questions to interact with the model.

Creative Writing: Continue a Story

Students can provide a starting sentence, and the model can continue the story. This demonstrates the model's ability to generate creative and contextually relevant text.

Text Summarization

Students can input a longer piece of text, and the model can provide a concise summary. This is a practical application of language models for information extraction.

In []:

input_ids

```
{'input_ids': tensor([[ 2, 236778, 236771, 236778, 236825,
238322, 236743, 236810, 238174,
236743, 236778, 236800, 237613, 7534, 237077, 243067,
237553, 12743,
70891]]), device='cuda:0'), 'attention_mask': tensor([[1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]],
device='cuda:0')}
```

In []:

output_tokens

```
tensor([[ 2, 236778, 236771, 236778, 236825, 238322, 236743,
236810, 238174,
236743, 236778, 236800, 237613, 7534, 237077, 243067,
237553, 12743,
70891, 124323, 237754, 237272, 238229, 99795, 237272,
191206, 237293,
127318, 77027]]), device='cuda:0')
```

```
In [ ]: vocab = tokenizer.get_vocab()

print(f"Total vocabulary size: {len(vocab)} tokens")

print("\nFirst 20 items in the vocabulary:")
# Print the first 20 key-value pairs to avoid overwhelming the output
for i, (token, token_id) in enumerate(vocab.items()):
    if i >= 20:
        break
    print(f"Token: '{token}', ID: {token_id}")
```

Total vocabulary size: 262145 tokens

First 20 items in the vocabulary:

Token: '__extranjero', ID: 157177

Token: 'السنة', ID: 213567

Token: 'dert', ID: 223184

Token: 'ABOUT', ID: 80309

Token: 'यूट्यूब', ID: 99100

Token: 'Assemblies', ID: 123655

Token: '__pains', ID: 49286

Token: '__representando', ID: 233637

Token: 'S̄O', ID: 228504

Token: '٠0٩٥٠', ID: 130370

Token: '__asked', ID: 4733

Token: '__interaction', ID: 9553

Token: 'SError', ID: 166031

Token: 'jszip', ID: 231938

Token: '镍', ID: 246141

Token: 'icha', ID: 44446

Token: '__analysing', ID: 86193

Token: 'UK', ID: 17827

Token: 'Utah', ID: 155063

Token: '__approvals', ID: 85745